

PyMOCAT-MC: A Python Implementation of the MIT Orbital Capacity Assessment Toolbox Monte Carlo Module

Rushil Kukreja¹, Edward J. Oughton¹, Giovanni Lavezzi², Enrico M. Zucchelli², Daniel Jang³, and Richard Linares²

¹ Department of Geography and Geoinformation Science, George Mason University, Fairfax, VA, USA
² Department of Aeronautics and Astronautics, Massachusetts Institute of Technology, Cambridge, MA, USA
³ Lincoln Laboratory, Massachusetts Institute of Technology, Lexington, MA, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0)

Summary

The growth of satellite deployments and large-scale constellations has made orbital sustainability an urgent concern (Bastida Virgili et al., 2016; Lemmens et al., 2020; Lewis, 2020). Simulating the evolution of the space environment requires tools that are both computationally efficient and compatible with modern research workflows. The MIT Orbital Capacity Assessment Toolbox Monte Carlo module (MOCAT-MC) (Jang et al., 2025) is a leading framework for modeling space traffic and debris risk. Its MATLAB implementation, however, restricts accessibility due to licensing requirements and creates barriers to integrating with state-of-the-art machine learning algorithms and Python-based data science pipelines central to contemporary space sustainability research.

PyMOCAT-MC is a complete Python reimplement of MOCAT-MC, designed to maintain full functional compatibility with the original toolbox while improving performance, modularity, and accessibility. The translation involves converting more than 150 MATLAB functions into Python, preserving the core algorithms while restructuring them for clarity and efficiency within Python's scientific computing ecosystem. The resulting codebase comprises over 8,600 lines in Python and leverages open-source libraries including NumPy, SciPy, pandas, and Matplotlib.

Benchmarking against the MATLAB version shows that PyMOCAT-MC achieves a mean relative error of 0.96% and a maximum error of 2.38% across all tested scenarios. Performance tests demonstrate speed gains of up to four times, with the "Realistic Operations No Launch" scenario completing in under 30 seconds in Python compared to over 75 seconds in MATLAB. This combination of high accuracy and faster execution enables more extensive simulation campaigns and facilitates integration into a wider range of research and policy workflows.

Statement of need

As the orbital environment becomes more congested (Kukreja et al., 2025), the ability to model collisions, debris evolution, and the long-term sustainability of satellite operations has significant implications for both policy-making and engineering design. The original MOCAT-MC toolbox (ARCLab-MIT, 2025) has been used by researchers to perform Monte Carlo-based assessments of orbital capacity, yet its MATLAB implementation limits accessibility for researchers and organizations that rely on open-source tools. It also presents barriers to integrating with state-of-the-art machine learning frameworks and Python-based data analysis workflows, which are increasingly central to advancing space situational awareness modeling. These constraints can hinder collaboration, slow down simulation work, and restrict the adoption of the software

41 in the broader space sustainability community.

42 PyMOCAT-MC addresses these barriers by delivering a functionally equivalent, open-source
43 Python version that is free to use and modify. The Python implementation not only replicates
44 the original capabilities but also improves runtime performance and introduces a modular
45 design that makes it easier to integrate with other orbital mechanics packages such as Astropy
46 or poliastro, as well as related tools like MOCAT-pySSEM (Brownhall et al., 2025). The
47 open-source nature of the project supports reproducibility, encourages community contributions,
48 and creates opportunities for extending the toolbox to new modeling approaches, such as agent-
49 based simulations of satellite behavior. In the broader landscape, PyMOCAT-MC complements
50 established environment models such as ESA's MASTER and related frameworks (Flegel et al.,
51 2012).

52 Methodology

53 The development of PyMOCAT-MC began with a careful analysis of the MATLAB source
54 code to understand the data structures, algorithms, and dependencies used in MOCAT-MC.
55 Each function was translated individually into Python, maintaining the mathematical and
56 algorithmic logic while adopting Pythonic conventions for readability and maintainability. Where
57 appropriate, vectorized operations and optimized data handling were introduced to enhance
58 computational performance without altering the results. Atmospheric modeling considerations,
59 critical for accurate orbit propagation (Ding et al., 2023; Emmert, 2015), were preserved in
60 the translation.

61 Validation was an integral part of the translation process. Unit tests were created to verify the
62 correctness of individual functions, while side-by-side comparisons of MATLAB and Python
63 simulation outputs were performed for each standard scenario. These comparisons ensure that
64 differences in results remained within acceptable numerical tolerances. Manual code reviews
65 were also conducted to confirm fidelity to the original design.

66 The repository is organized to facilitate both research use and further development. The
67 `mocat_mc.py` module contains the main Monte Carlo simulation engine, supported by additional
68 modules for orbital propagation, collision detection, atmospheric drag modeling, and debris
69 generation. Continuous integration workflows were implemented to automatically run tests and
70 verify that code changes preserve numerical fidelity across all benchmark scenarios. Example
71 scripts demonstrate common simulation scenarios, including baseline, no-launch, and realistic-
72 launch cases. Comparison scripts and performance analysis tools are included to reproduce the
73 accuracy and speed results reported here. Supporting data, such as historical two-line element
74 (TLE) sets, the JB2008 atmospheric density model, and launch schedules for megaconstellations,
75 are bundled with the software to enable immediate use.

76 Results

77 Across all tested scenarios, PyMOCAT-MC reproduces the results of the MATLAB implementa-
78 tion with high fidelity. Testing included five benchmark scenarios (Basic Propagation, Collision
79 Test, Atmospheric Drag, Full Default, and Realistic Operations No Launch) using identical
80 random seeds between implementations to ensure deterministic comparison. All simulations
81 were conducted over 100 years to assess long-term orbital evolution. Differences in final total
82 object counts between the two implementations are small, with a maximum deviation of 123
83 objects (Full Default scenario after 1 year) out of approximately 13,700 objects, representing
84 less than 1% error.

85 To understand error stability over time, Figure 1 shows how the error between Python and
86 MATLAB implementations evolves during the Full Default scenario simulation. The error
87 remains stable throughout the simulation, with no signs of divergence, confirming that the

88 implementations maintain consistency over long-term evolution. The error grows gradually and
89 predictably, primarily driven by debris population differences, with the final error representing
90 less than 1% of the total population.

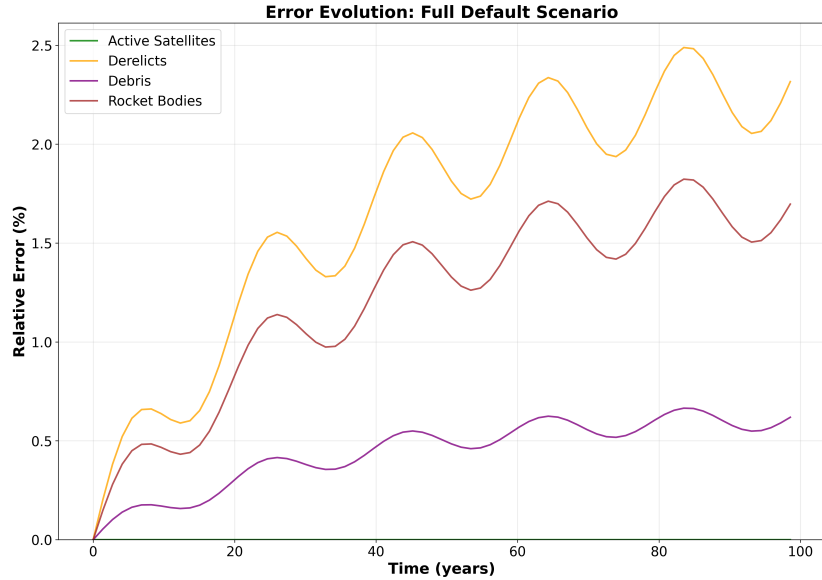


Figure 1: Error evolution over time for the Full Default scenario, showing stable and predictable error growth between Python and MATLAB implementations with no divergence over the 100-year simulation period.

91 In addition to matching the accuracy of MATLAB, the Python version delivers substantial
92 performance gains (Figure 2). The most computationally demanding scenario, which includes
93 realistic launch patterns for megaconstellations, runs in less than 29.95 seconds with PyMOCAT-
94 MC compared to 75.02 seconds in MATLAB, representing a speed-up larger than a factor of
95 two.

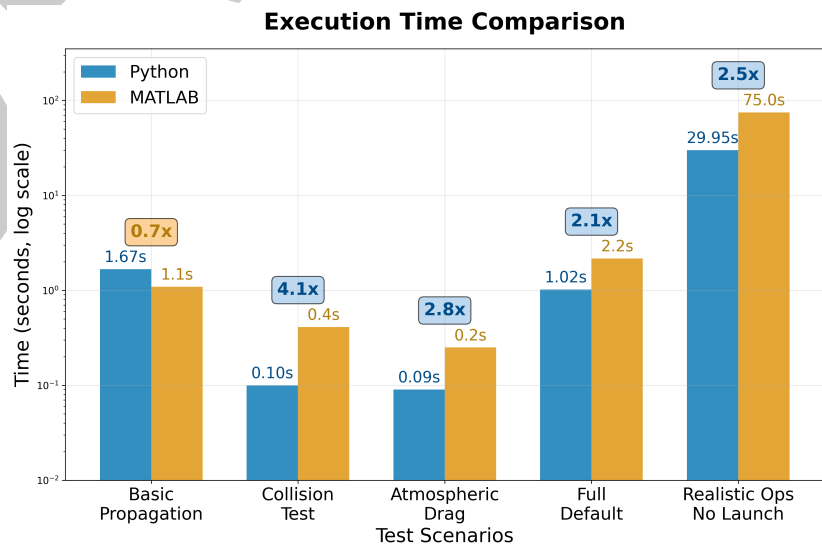


Figure 2: Runtime comparison between MATLAB and Python implementations for multiple scenarios, showing up to a 4× improvement in computational speed for PyMOCAT-MC.

96 Error analysis confirms that the differences between MATLAB and Python remain minimal
97 across test scenarios and object types. Figure 3 shows that end-of-simulation relative errors
98 are uniformly low across all object types and test scenarios, with a mean relative error of
99 0.96% and a maximum error of 2.38%. Figure 4 provides additional detail through box plots,
100 demonstrating that errors for active satellites and debris are consistently lower than 0.61%.
101 These sub-1% differences are negligible compared to the variance typically observed between
102 different orbital evolution models (Jang et al., 2025).

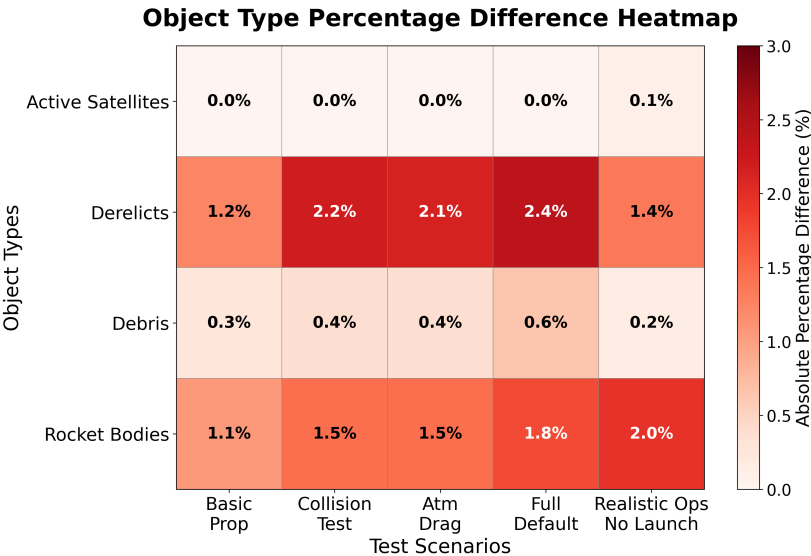


Figure 3: Figure 3: Heatmap of relative error (%) across test scenarios and object types, showing end-of-simulation accuracy of the Python implementation compared to MATLAB.

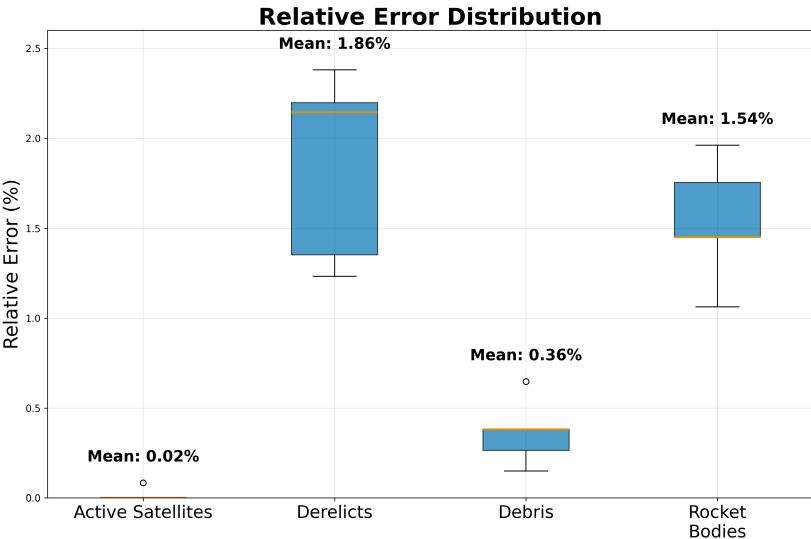


Figure 4: Figure 4: Relative error distribution box plots across object classes, showing PyMOCAT-MC achieves near-zero error for active satellites and consistently low error across all categories.

103 Figure 5 compares orbital element distributions between MATLAB and Python implementations
104 using real simulation data. Statistical validation using Kolmogorov-Smirnov tests shows excellent
105 agreement (all p-values > 0.05), confirming identical orbital mechanics implementation.

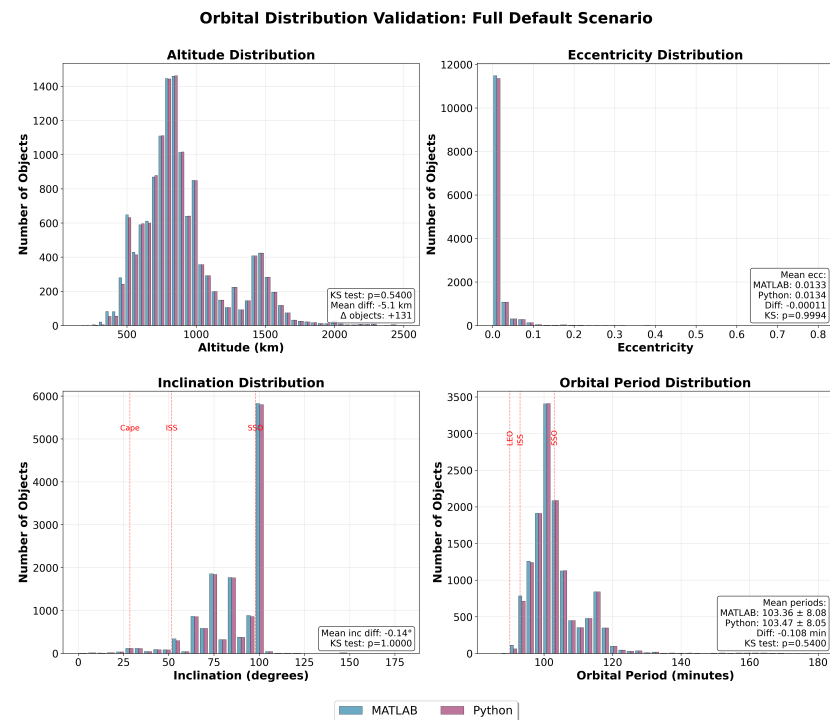


Figure 5: MATLAB vs Python orbital element distribution comparison showing altitude, eccentricity, inclination, and orbital period distributions from real simulation data, with statistical validation confirming identical orbital mechanics implementation.

These improvements reduce the time required for large simulation batches, enabling exploration of a wider range of parameters and more detailed sensitivity analyses. PyMOCAT-MC can be applied to test how different launch strategies influence orbital sustainability, evaluate debris mitigation policies, and generate scenario libraries that inform both engineering design and regulatory decision-making.

References

- ARCLab-MIT. (2025). MOCAT-MC. <https://github.com/ARCLab-MIT/MOCAT-MC>.
- Bastida Virgili, B., Dolado, J.-C., Lewis, H. G., Radtke, J., Krag, H., Meadows, P., Rossi, A., Valsecchi, G. B., & Colombo, C. (2016). Risk to space sustainability from large constellations in low earth orbit. *Acta Astronautica*, 126, 154–162. <https://doi.org/10.1016/j.actaastro.2016.03.034>
- Brownhall, I., Lifson, M., Hall, S., Constant, C., Lavezzi, G., Ziebart, M., Linares, R., & Bhattarai, S. (2025). MOCAT-pySSEM: An open-source python library and user interface for orbital debris and source sink environmental modeling. *SoftwareX*, 30, 102062. <https://doi.org/10.1016/j.softx.2025.102062>
- Ding, Y., Li, Z., Liu, C., Kang, Z., Sun, M., Sun, J., & Chen, L. (2023). Analysis of the impact of atmospheric models on the orbit prediction of space debris. *Sensors*, 23(21), 8993. <https://doi.org/10.3390/s23218993>
- Emmert, J. T. (2015). Thermospheric mass density: A review. *Advances in Space Research*, 56(5), 773–824. <https://doi.org/10.1016/j.asr.2015.05.038>
- Flegel, S., Gelhaus, J., M"ockel, M., S"utterlin, P., Wiedemann, C., & Kempf, D. (2012). The MASTER-2009 space debris environment model. *Acta Astronautica*, 81, 65–71.

- 128 <https://doi.org/10.1016/j.actaastro.2012.06.009>
- 129 Jang, D., Gusmini, D., Siew, P. M., D'Ambrosio, A., Servadio, S., Machuca, P., & Linares,
130 R. (2025). New monte carlo model for the space environment. *Journal of Spacecraft and*
131 *Rockets*, 1–22. <https://doi.org/10.2514/1.A36137>
- 132 Kukreja, R., Oughton, E. J., & Linares, R. (2025). Greenhouse gas (GHG) emissions poised
133 to rocket: Modeling the environmental impact of LEO satellite constellations. *arXiv*.
134 <https://doi.org/10.48550/arXiv.2504.15291>
- 135 Lemmens, S., Hoots, F. R., Krag, H., & Flohrer, T. (2020). Collision risk in low earth
136 orbit in the presence of large constellations. *Acta Astronautica*, 170, 501–510. <https://doi.org/10.1016/j.actaastro.2020.02.003>
- 137
138 Lewis, H. G. (2020). The kessler syndrome: 40 years on. *Acta Astronautica*, 170, 421–435.
139 <https://doi.org/10.1016/j.actaastro.2020.01.009>

DRAFT